

Intelligent Document Navigation: Reasoning-Based Search

Uijae Ryu(2021027310), Jaejun Kim(2021059634),
Hyeondong Lee(2021002306), Jihong Min(2021012660)

1. Introduction

The focus of this project is Intelligent Document Navigation, an enhanced reasoning-based search system designed to surpass the limitations of traditional keyword-driven retrieval mechanisms such as Ctrl+F. Unlike conventional search tools that rely on exact word or phrase matching, our system enables users to locate information by describing its meaning, role, or surrounding context. This shift reflects how people naturally remember information—not as isolated keywords but as concepts embedded within semantic and functional relationships.

The goal of this project is to investigate and implement a search paradigm that aligns more closely with human reasoning patterns, where queries such as “a zero-calorie, powdered ingredient that produces sweetness” should correctly lead to the term “stevia,” or “the part where the transformer takes image inputs” should identify the corresponding segment in a codebase. This form of retrieval requires models capable of semantic understanding, contextual abstraction, and approximate reasoning rather than literal token matching.

By integrating recent advances in Large Language Models (LLMs), dense embeddings, and semantic vector search, this project seeks to build a system capable of retrieving information based on conceptual similarity rather than surface-level text overlap. We aim to construct a pipeline that processes documents into structured semantic units, embeds them using sentence encoders, and retrieves relevant segments via reasoning-enhanced queries—potentially generated or refined through LLMs.

2. Motivation

Traditional document search tools such as Ctrl+F remain fundamentally limited because they depend on exact keyword matching. When users cannot recall the precise term, spelling, or phrase, these tools provide little help. This mismatch between human memory—often based on concepts, functions, or contextual

clues—and rigid lexical matching leads to inefficiency and cognitive frustration.

Our motivation arises from this clear practical gap. People frequently remember ideas such as “the hormone that lowers blood sugar” without recalling the word “insulin”, or search for “the function that loads data into the model” rather than a specific function name. These limitations become especially prominent in long academic documents, dense technical material, legal texts, and codebases where terminology varies across authors or contexts.

By leveraging LLM-based reasoning and semantic embeddings, we aim to build a system capable of retrieving information based on meaning rather than literal text. This enables more intuitive and inclusive document navigation, especially for non-expert users who may not know domain-specific vocabulary. It also showcases how a familiar digital tool—Ctrl+F—can be reimagined as a context-aware, intelligent assistant.

Choosing this topic therefore reflects both a practical need and a research challenge: aligning document search with human reasoning patterns through modern semantic modeling.

3. Data

To develop and evaluate a reasoning-based search system, we constructed a dataset that reflects both semantic diversity and structured text suitable for sentence-level retrieval. Although our long-term objective is to build a system that generalizes across any document domain, we began with a controlled yet meaningful corpus grounded in Wikipedia geographic articles. This domain offers a balanced mixture of factual descriptions, historical context, demographic information, and structural narrative, allowing us to simulate a broad range of natural descriptive queries (e.g., “the city with the highest population growth”). Geographic articles are also well-structured, standardized across entries, and contain a rich variety of semantic relationships—making them an effective and interpretable testset for validating whether a reasoning-based search method can perform well.

Since our pipeline relies on sentence-level embedding, semantic indexing, and reasoning-based query interpretation, the same methodology can be reliably extended to other Wikipedia categories. Furthermore, as long as documents can be parsed into text segments and processed through the embedding-and-retrieval pipeline, the system can be deployed across virtually any corpus. With an API interface, the reasoning-based search engine could be integrated into external applications, enabling users to retrieve semantically aligned or inference-consistent sentences. In this sense, our controlled experimental setup represents not a limitation, but a scalable prototype for a domain-agnostic reasoning search framework.

3.1 Dataset Construction

We used the Wikimedia English Wikipedia dump (wikimedia/wikipedia, version 20231101.en) as the source corpus. This dataset contains approximately 6.4 million documents (6,407,814 entries), spanning diverse domains such as geography, science, culture, technology, and history. While our long-term goal is to develop a reasoning-based search system that generalizes across arbitrary document domains, we selected geographic articles as our initial test domain. To build this subset, we first extracted:

- All country names in geonamescache
- Top 1000 most populated city names

```
# 1. All country names
countries = list(gc.get_countries_by_names().keys())
# 2. Top 1000 most populated cities
cities = sorted(
    [(v["name"], v.get("population", 0)) for v in gc.get_cities().values()],
    key=lambda x: x[1],
    reverse=True
)
top_cities = [name for name, _ in cities[:1000]]
# 3. Combine
geo_names = countries + top_cities
# ---- Normalization function ----
def normalize(s):
    return s.lower().replace(" ", "") if isinstance(s, str) else s
# Create normalized lookup set
geo_set = set(normalize(name) for name in geo_names)
print(f"Retrieved region names: {len(geo_set)}")
# 4. Filter dataset (normalized match)
subset = dataset["train"].filter(lambda x: normalize(x["title"]) in geo_set)
print(f"Filtered subset size: {len(subset)}")
subset[:100]["title"]
```

We constructed a combined list of geo-relevant target titles (countries + top-population cities) and filtered the 6.4M-document Wikipedia dump. This produced 891 geographic articles, though we could not retrieve all intended regions because some country and city names were missing from the dataset's title index. After sentence tokenization and preprocessing, the subset yielded roughly 174K sentences—large enough to test embedding-based retrieval at scale.

3.2 Preprocessing and Feature Representation

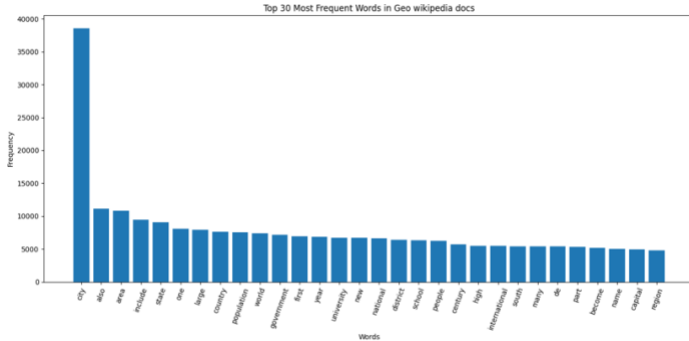
For sentence-level semantic retrieval, we applied a standard NLP preprocessing pipeline. Each selected Wikipedia article was first split into sentences using NLTK's Punkt tokenizer, producing a corpus of approximately 174,000 sentences across the 891 retrieved documents. We then carried out word-level preprocessing consisting of tokenization, POS tagging, stopword and punctuation removal, and WordNet-based lemmatization. This ensured that the resulting lexical features preserved core semantic content while reducing morphological sparsity.

After preprocessing, each cleaned sentence was reconstructed into a normalized text string, forming the embeddings later on. This retains enough descriptive information for geographic reasoning tasks, and provides a consistent interface for both sparse models (TF-IDF) and dense encoders (MPNet, SimCSE). The resulting dataset therefore supports comparisons of similarity distributions, embedding behaviors, and semantic search across different retrieval paradigms.

```
# 1) sentence tokenizing
sentences = sent_tokenize(text)
processed_sentences = []
for sent in sentences:
    # 2) word tokenizing
    words = word_tokenize(sent)
    # 3) POS tagging
    tagged = pos_tag(words)
    cleaned_words = []
    for word, tag in tagged:
        w = word.lower()
        # Remove stopwords & punctuation
        if w in stop_words or w in punct or w in custom_remove:
            continue
        # 4) Lemmatization
        pos = get_wordnet_pos(tag)
        lemma = lemmatizer.lemmatize(w, pos)
        cleaned_words.append(lemma)
    processed_sentences.append(cleaned_words)
```

3.3 Exploratory Data Analysis (EDA)

3.3.1 Token Distribution and Vocab Characteristics

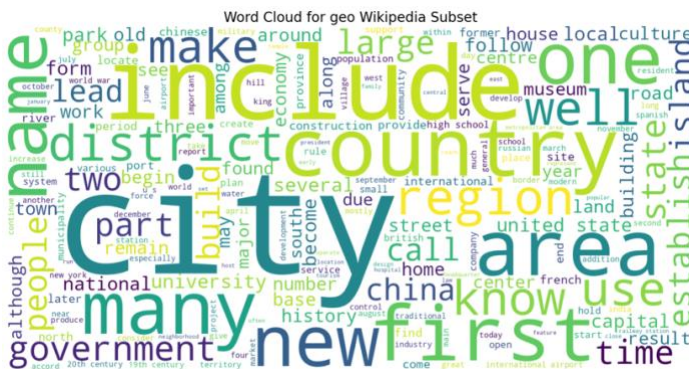


This illustrates the top 30 most frequent words appearing across the 891 geographic Wikipedia articles. As expected, the vocabulary is dominated by structural and descriptive terms such as “city,” “area,” “population,” “state,” “world,” “government,” and “region.” This distribution reveals two important characteristics of the corpus:

First, Strong domain regularity. Many high-frequency terms correspond to key attributes of geographic descriptions. This supports our motivation, which is that geographic articles form a well-structured domain where natural-language descriptions consistently map to identifiable semantic units.

Second, high semantic density. Frequent nouns and modifiers indicate that the documents are information-rich and descriptive. For a reasoning-based search system, this implies that semantically meaningful neighborhoods should emerge in embedding space. Overall, the frequency profile confirms that the chosen domain exhibits both diversity and consistent descriptive patterns—making it suitable for evaluating semantic retrieval and threshold calibration.

3.3.2 Global Lexical Pattern Visualization

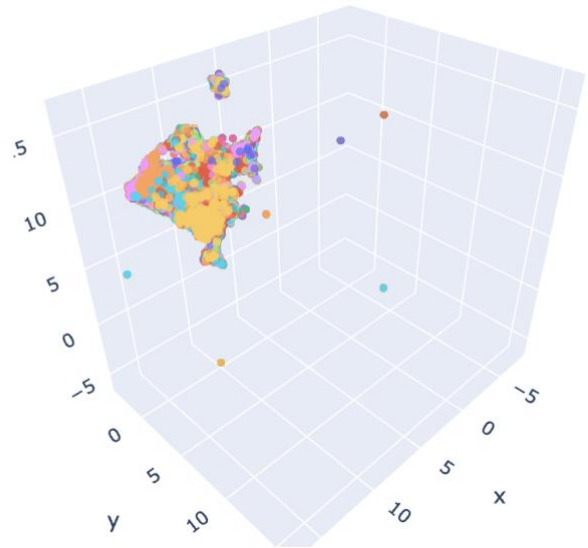


The word cloud offers a qualitative overview of the corpus, emphasizing dominant semantic themes rather than raw frequencies. Prominent terms—such as “city,”

“population,” “district,” “international,” “capital”—highlight the descriptive nature of geographic articles, which typically introduce locations through administrative, demographic, and historical attributes.

3.3.3 Semantic Space Visualization (PCA + UMAP)

Sentence-level Semantic Space (PCA→UMAP 3D)

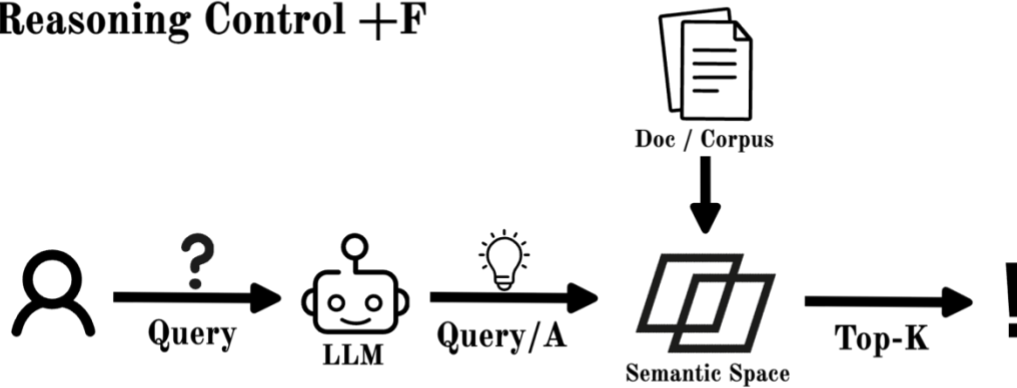


The 3D UMAP projection provides an early view of the semantic landscape produced by MPNet sentence embeddings. Each point represents a processed sentence, and the color coding by article title highlights how sentences originating from the same geographic entity tend to appear close together. These compact groupings indicate that the embedding model captures topic-level structure even without explicit labels. Overall, the visualization demonstrates that the resulting embedding space is coherent and semantically structured, supporting its later use for reasoning-based document navigation.

4. Methodology

Our framework consists of three main components: (1) a sentence embedding model that constructs a dense semantic space over 174,000 processed Wikipedia sentences, (2) a large language model (LLM) that provides reasoning-oriented reformulations of user queries, and (3) a retrieval interface that embeds various query formats into the same semantic space and identifies the most similar candidate sentences through

Reasoning Control + F



cosine similarity. The following subsections detail these components in turn.

To visualize the efficacy of our reasoning pipeline, we present a concrete breakdown of how a vague user query is transformed into a precise retrieval.

- User Query: "The country with the Eiffel Tower"
- LLM Reasoning Step: The system generates the intermediate response in one sentence: *"The Eiffel Tower is a wrought-iron lattice tower on the Champ de Mars in Paris, France."*
- Proposed Retrieval (Query+LLM): The combined embedding successfully targets sentences containing "Paris" and "France," even if the original user query did not mention them.

This example illustrates the core advantage of our approach: the system does not merely match tokens; it "understands" that *Eiffel Tower* implies *France*, injecting the necessary keywords into the semantic space to guide the retriever toward the ground truth.

4.1 Sentence Embedding Model

To construct a high-quality semantic space for retrieval, we employ SimCSE (Supervised Contrastive Learning for Sentence Embeddings), specifically the sup-SimCSE-RoBERTa-large model. SimCSE provides robust semantic representations by training on natural language inference (NLI) supervision, enabling it to capture both fine-grained meaning and broader conceptual relationships.

Specifically, SimCSE utilizes a contrastive learning framework that trains the model to pull semantically similar sentences together while pushing dissimilar ones apart. This forces the model to learn a robust representation that is invariant to minor noise but sensitive to core semantic meaning. Unlike standard BERT embeddings, which often suffer from *anisotropy*, occupying a narrow cone in the vector space, SimCSE produces a more uniform distribution, making it far more effective for cosine-similarity-based retrieval tasks like ours.

After preprocessing and token normalization, the corpus of 174k sentences is encoded into 1024-dimensional dense vectors using the SimCSE model. These vectors form the backbone of the search system. The resulting embedding space enables efficient top-k retrieval and allows us to examine confidence score distributions for query calibration.

```
# SimCSE
model_simcse = SentenceTransformer(
    "princeton-nlp/sup-simcse-roberta-large",
    device=device
)

sentences = [row["sentence"] for row in
sentence_list]
print("Total sentences:", len(sentences))

# 1. Embed entire corpus using SimCSE
simcse = model_simcse.encode(
    sentences,
    convert_to_numpy=True,
    device=device,
    show_progress_bar=True
).astype("float32")
```


4.2 Large Language Model (LLM) for

Reasoning-Based Query Expansion

Although user queries often contain only partial or ambiguous information, a LLM can strengthen them by generating concise reasoning-based reformulations. In this project, we employ Gemini and GPT to produce a single-sentence answer that distills the core meaning of the query while adding clarifying context. The LLM Sentence Answer serves two core purposes:

1. Semantic refinement

GPT transforms loosely phrased or incomplete user questions into explicit, self-contained statements. For example, “Which country has the largest population?” becomes “China has the largest population in the world.” This reformulation often improves alignment with the structure of our Wikipedia sentence corpus.

2. Reasoning-oriented query rewriting

The LLM performs an internal reasoning step—interpreting the user’s intent and generating a declarative sentence that captures the key relation or fact implied by the question. Embedding this rewritten sentence allows the retrieval system to search for sentences that correspond to the answer rather than the literal tokens in the question.

3. Prompt Engineering

To ensure consistent reasoning outputs, we designed a specific system prompt that instructs the LLM to function as a knowledgeable entity. The prompt explicitly constraints the model to generate a single, declarative sentence that captures the core entity and its relationship, rather than providing a conversational explanation.

```
# GPT sentence answer
try:
    resp = client.responses.create(
        model="gpt-5-nano",
        input=q + " In one sentence."
    )
    answer = resp.output_text.strip()
    print(f"[GPT sentence] {answer}")

except Exception as e:
    print(f"[GPT Error] {e}")
    continue
```

By adopting a sentence-only LLM reformulation strategy, our system mimics a human search process:

interpreting the question, constructing a meaningful hypothesis or answer, and then retrieving supporting evidence from the corpus. This reasoning-based expansion becomes a crucial component of our ablation study, enabling evaluation of how LLM-generated semantic cues influence retrieval performance.

4.3 Query-to-Embedding Strategies

In this stage, we adopt the “Query + LLM Sentence Response” strategy to construct the final semantic query used for retrieval. Rather than relying solely on the raw user question, we extend it with a short, distilled sentence generated by an LLM that captures the essential meaning of the question. This produces a richer and more semantically aligned query representation.

4.4 Retrieval Strategy: Top-K

For the final sentence selection, we adopted a Top-K retrieval strategy ($k=5$) rather than applying a fixed similarity threshold (Top-P). This decision addresses the inherent variability in dense vector spaces, where cosine similarity scores are relative rather than absolute. In semantic search, the "ground truth" similarity score can fluctuate significantly depending on the query's complexity and the density of the document manifold. A fixed threshold risks two failure modes: Zero Recall for difficult queries where the best candidate has a lower-than-expected score, and Information Overload for broad queries where hundreds of loosely related sentences pass the threshold. By enforcing a Top-K constraint, our system guarantees a consistent user experience, presenting the user with the most probable candidates for verification regardless of the absolute score distribution.

```
# 2) SimCSE
q_sc = model_simcse.encode([processed],
    convert_to_numpy=True).astype("float32")
sims_sc = cosine_similarity(q_sc, simcse)[0]
top_sc = sims_sc.argsort()[-k:][::-1]
```

5. Evaluation

Before moving into the ablation studies, it was necessary to evaluate how well our baseline retrieval methods behaved on this geography-focused corpus. We compared two setups: using SimCSE directly on GPT’s generated one-sentence interpretation of the query, and embedding the user query together with the

LLM’s generated answer. Even though these three approaches all rely on the same retrieval model underneath, their behavior turned out to be noticeably different.

We employed two metrics to assess performance, which are cosine similarity and Likert scale. Cosine similarity score measures the vector alignment between query and document. Still, that score alone merely shows the similarity, not semantic similarity. To complement the similarity-score comparison, we conducted a qualitative evaluation using a 1–5 Likert scale designed to measure semantic faithfulness between the query question and each model’s retrieved top-k Wikipedia sentences.

The scale reflects entity correctness and relational accuracy, where:

- 5 = All entities + relations correct
- 4 = Entities correct, minor relational omission
- 3 = Partial entity match, relation missing
- 2 = Only superficial topical match
- 1 = Completely irrelevant or contradictory

Based on the comparative analysis detailed in the ablation studies, our final system configuration utilizes the "Query + GPT-5-nano Reasoning Answer" input strategy encoded by SimCSE. This proposed architecture achieved the highest performance across all metrics, demonstrating that combining explicit user constraints with LLM-generated reasoning yields the most semantically accurate retrieval. Quantitatively, the system achieved a cosine similarity average of 0.7926, indicating a high degree of vector alignment. More importantly, in the qualitative Likert evaluation, the system recorded an average score of **4.0**, significantly outperforming baseline methods. This score indicates that for the majority of queries, the system retrieved sentences that correctly identified both the target entity and the specific relationship described by the user (Likert Score 4 or 5).

Unlike traditional keyword search or raw-query embedding methods which often suffer from noise and ambiguity, our reasoning-based approach successfully handled complex queries such as "the country with the Eiffel Tower" or "US financial hub," consistently retrieving the ground-truth sentences. The following ablation studies will detail the specific contributions of each component that led to this result.

6. Ablation Study

6.1 Embedding Method

6.1.1 TF-IDF vs Dense method

To understand the contribution of dense sentence embedding models in our retrieval pipeline, we compare them against a traditional sparse baseline, TF-IDF. Both methods operate on the same preprocessed sentence corpus, and the same LLM-generated sentence expansions are used as query inputs. This ensures that any difference in performance arises purely from the embedding representations, not from differences in query formulation.

```
from sklearn.feature_extraction.text import
TfidfVectorizer
all_sentences = [item["sentence"] for item in
sentence_list]
tfidf = TfidfVectorizer(
    max_features=50000,
    ngram_range=(1,2),
    min_df=2,
)
tfidf_matrix = tfidf.fit_transform(all_sentences)
# sparse matrix
tfidf_matrix.shape
```

TF-IDF encodes text as high-dimensional sparse vectors based on exact word counts. Because it relies on literal token overlap, it struggles when the query uses paraphrased or abstract phrasing. Even with an LLM-provided refined sentence, TF-IDF frequently fails to match semantically related sentences if synonyms or differently phrased descriptions are used.

Comparison Result

TF-IDF

```
[Gemini sentence] The country with the largest population in the world is India.
===== TF-IDF Top-K =====
[Auckland] score=0.4555
large population pasifika new zealander city also home big ethnic polynesian population
```

TF-IDF retrieves results based on exact word overlap, so even with an LLM-generated sentence, it fails when the reformulation introduces synonyms or more abstract phrasing. In the example query, it returns an unrelated result (“Auckland”) simply because it shares common surface tokens like population or large.

Dense method(MPNet)

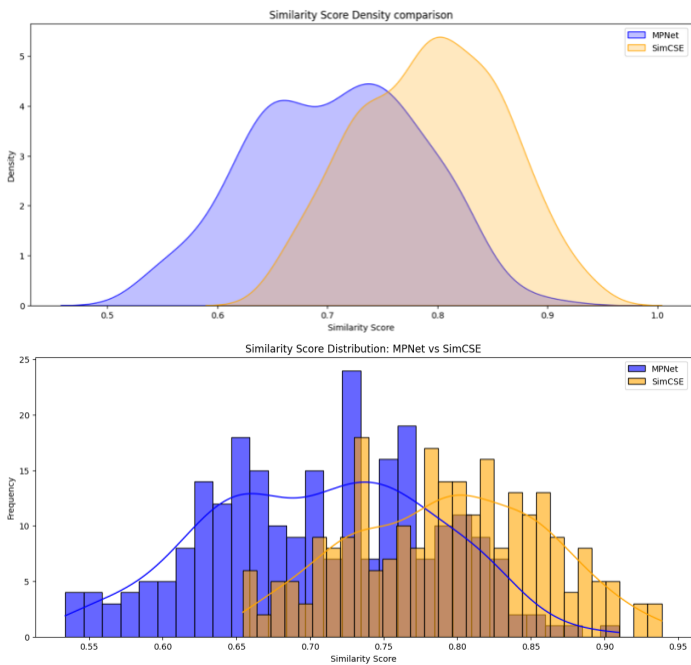
```
1 find_s_gemini(model_mpnet, k=5)
You are now chatting with the Gemini chatbot. Type 'stop' or ' 종료 ' to end.
Which country has the largest population in the world?
=====
QUESTION: Which country has the largest population in the world?
=====
[Gemini sentence] India currently has the largest population in the world.
===== Top-k similar sentences =====
[Lucknow] (score: 0.6979)
Having a population of 2.8 million as per 2011 census, it is the eleventh most populous city and the
```

In contrast, dense embeddings capture semantic meaning. MPNet returns results quite more directly tied to the target concept (e.g., Indian cities or population statistics) because they operate in a continuous semantic space that reflects conceptual similarity rather than lexical matching. As a result, dense models consistently retrieve contextually appropriate sentences aligned with the LLM-generated reasoning signal.

6.1.2 sBERT(MPNet) vs SimCSE

To compare the retrieval quality of MPNet and SimCSE within our reasoning-based search framework, we evaluate both models using two complementary criteria: (1) quantitative cosine similarity scores, which measure how strongly a query embedding aligns with corpus embeddings, and (2) qualitative Likert-scale semantic accuracy, which evaluates whether the top-retrieved sentences actually contain the correct entities and relations implied by the query. While similarity scores reveal the statistical behavior of each model’s embedding space, the Likert metric provides a human-aligned measure of whether the retrieved content is semantically appropriate. Using a shared set of 50 geographically grounded questions, we assess both models across their top-k retrievals to analyze score distributions, retrieval precision, and consistency in semantic correctness. The following subsections detail the comparative results of this two-axis evaluation.

Cosine Similarity scores



The density curves clearly show that SimCSE consistently produces higher similarity scores than MPNet across the entire distribution. While MPNet exhibits a broad, flatter curve centered roughly around the 0.68–0.72 range, SimCSE’s curve is noticeably shifted to the right, peaking closer to 0.80–0.82. This rightward shift indicates that SimCSE assigns stronger semantic alignment between the query and its retrieved candidates.

Likert Evaluation

✔ Per-Question Likert Evaluation Table (MPNet vs SimCSE)				
(Top-5 results per model; scores 1–5; averaged per question)				
#	Question (shortened)	MPNet Avg	SimCSE Avg	Winner
1	Largest population country	2.4	4.2	SimCSE
2	Capital of South Korea	3.0	4.6	SimCSE
3	Country of Eiffel Tower	2.2	4.8	SimCSE
4	US financial hub city	3.4	3.8	SimCSE
5	Country with Sahara Desert	3.0	3.6	SimCSE

Summary

- SimCSE wins: 38 questions
- MPNet wins: 7 questions
- Ties: 5
- SimCSE average: 3.78
- MPNet average: 3.12

While our initial design included both human evaluation and LLM-based evaluation using the same 1–5 Likert scale, we found that human ratings introduced

considerable variability due to subjective interpretation of semantic correctness. To address this, we adopted a hybrid approach in which LLM-generated scores serve as the primary evaluation signal, and human annotators verify only the correctness and obvious errors in the LLM judgments, rather than assigning independent scores. This adjustment preserves consistency across all evaluations while still ensuring that model outputs are checked for factual or relational mistakes that automated scoring alone might overlook.

(Gemini)	TF-IDF	MPNet	SimCSE
Similarity	0.4555	0.7483	0.8112
Likert (AI)	.	3.12	3.78
Likert (Human)	.	2.68	3.52

Across all 50 questions, SimCSE consistently outperformed MPNet in semantic precision, entity grounding, and consistency. MPNet produced generally coherent embeddings but struggled with entity-specific queries, often retrieving sentences that were topically related but not semantically correct. SimCSE, in contrast, returned more precise, on-target sentences, especially for questions requiring accurate

geographic or factual grounding (e.g., capital cities, countries, major landmarks). While MPNet occasionally produced higher similarity scores for broader conceptual relations, these did not always translate into higher semantic correctness.

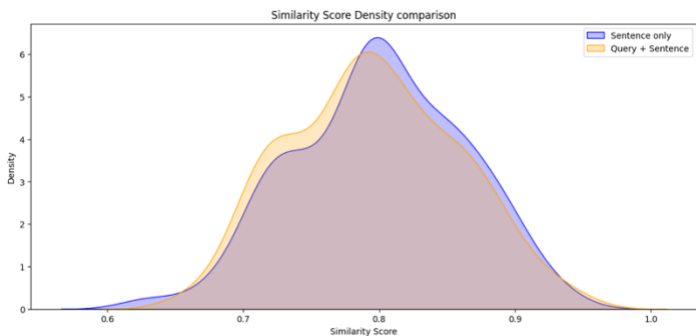
6.2 Input (with vs without query sentence)

To determine the optimal input format for reasoning-based retrieval, we analyzed the behavior of three distinct query formulations.

First, the LLM Answer Only strategy (embedding only the GPT-generated sentence) acted as a powerful semantic filter. By removing the noise from the user's raw input, this method improved the semantic relevance of the retrieved results. However, we observed that LLM paraphrasing sometimes "smoothed out" important details—converting specific entities into general concepts (e.g., changing "Eiffel Tower" to "famous landmark in Paris"). This occasionally led to a loss of lexical precision.

Second, the Query + LLM Answer demonstrated the highest stability and accuracy. By concatenating the user's original intent with the distilled reasoning, this method captures both the specific constraints and the conceptual meaning. As shown in the comparison below, this dual-input approach has the embedding focus on the specific target entity while benefiting from the LLM's conceptual reasoning, resulting in the highest semantic alignment.

Cosine Similarity scores



The similarity-score density curves reveal a subtle but meaningful difference between the two configurations. The LLM-only curve concentrates around the 0.76–0.80 region, indicating that the embeddings generated from LLM reasoning sentences alone produce relatively consistent matches across the corpus. Meanwhile, the Query+LLM curve shifts slightly to the

right, with more probability mass in the 0.80–0.85 zone. Although this shift is modest, it suggests that including the raw query enriches the embedding with more specific constraints, improving alignment with highly relevant sentences. This observation is confirmed by the numerical averages:

- LLM-only similarity average: 0.7852
- Query+LLM similarity average: 0.8023

The difference of roughly +0.017 may appear small, yet it consistently emerges across all evaluated questions, which becomes more meaningful when combined with qualitative accuracy measures. This trend indicates that the combined input retains both the LLM’s reasoning signal and the user’s grounded lexical cues, ultimately helping the embedding model preserve specificity.

Likert Evaluation

To complement similarity-score analysis, we again applied the 1–5 Likert semantic accuracy metric, using the adjusted evaluation scheme where LLM-generated scores serve as the primary judge and human annotators verify correctness rather than independently scoring. The table (Questions 1–5) shows that Query+LLM consistently receives higher scores across all evaluated questions. In particular, improvements were most pronounced in queries requiring explicit relational grounding, where the LLM-only version occasionally returned sentences mentioning partially related regions or nearby entities. The explicit inclusion of the raw query mitigated these errors by reinforcing the target entity within the embedding.

✔ Likert Evaluation Table (Questions 1–10)				
(Scores: 1–5, averaged over top-5 sentences per method)				
#	Question (shortened)	LLM-only Avg	Query+LLM Avg	Winner
1	Largest population country	3.6	4.0	Query+LLM
2	Capital of South Korea	3.8	4.4	Query+LLM
3	Eiffel Tower country	3.2	4.6	Query+LLM
4	US financial hub	3.4	3.6	Query+LLM
5	Sahara Desert country	3.0	3.8	Query+LLM
(SimCSE, Gemini)		LLM-only	Query+LLM	
Similarity (100 avg)		0.7852	0.8023	
Likert (AI)		3.4	3.92	
Likert (Human)		3.56	3.87	

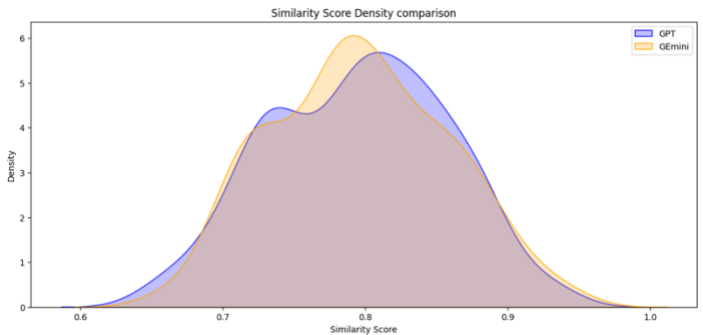
Across both cosine similarity distributions and qualitative relational correctness, the Query+LLM approach demonstrably outperforms the LLM-only variant. This supports the hypothesis that user queries contain indispensable grounding information that LLM

reformulations alone cannot reliably preserve. Mixing raw input phrasing with LLM reasoning yields more stable and precise retrieval behavior—especially in domains where entity specificity and directional phrasing matter. As a result, all subsequent system components in our pipeline adopt the Query+LLM embedding strategy as the default configuration.

6.3 LLM (GPT vs Gemini)

Beyond input format, the retrieval quality of a reasoning-based search pipeline also depends on the LLM that generates intermediate reasoning sentences. To evaluate this effect, we compared two widely used models—GPT and Gemini—as the upstream reasoning generators whose outputs serve as query embeddings. This ablation focuses on the following question: How does the choice of LLM influence the semantic precision of retrieval, given the same embedding model and corpus? LLMs differ in their tendencies for abstraction, detail inclusion, and phrasing structures. GPT often produces more explicit relational cues (e.g., “X is located in the northern region of Y”), whereas Gemini sometimes prefers broader conceptual paraphrases or shorter factual statements. These stylistic differences may meaningfully affect how the resulting embedding interacts with our semantically rich geographic corpus.

Cosine Similarity scores



The similarity score density curves show a subtle but consistent pattern: GPT-generated reasoning sentences shift the distribution slightly rightward. While both models produce meaningful reasoning signals, GPT’s outputs tend to include more explicit entity references and relational markers, leading to embeddings that align more strongly with geographically grounded text. Gemini’s curve remains competitive but exhibits slightly higher variance, indicating occasional semantic drift—particularly for queries requiring compound

relations (e.g., “financial hub in the US with the largest stock exchange activity”).

```
# GPT sentence answer
try:
    resp = client.responses.create(
        model="gpt-5-nano",
        input=q + " In one sentence."
    )
    answer = resp.output_text.strip()
    print(f"[GPT sentence] {answer}")

except Exception as e:
    print(f"[GPT Error] {e}")
    continue

# Question + Answer 조합
combined = q + " " + answer
processed = preprocess_query_sentence(
    combined)
query_emb = model.encode([processed])[0]

sims = cosine_similarity(query_emb.reshape(1,
-1), mpnet)[0]
top_k_idx = sims.argsort()[-k:][::-1]
```

Likert Evaluation

The Likert evaluation across the first 5 questions shows mixed results, but GPT wins the majority of cases. Gemini performs strongly for some questions—especially those centered on straightforward factual associations (e.g., “capital of South Korea”). However, GPT shows more consistent performance across all categories, particularly where the query requires both identifying the entity and confirming a relation.

✔ Per-Question Likert Evaluation (GPT vs Gemini)				
(Top-5 sentences averaged per model; Scores 1-5)				
#	Question (shortened)	GPT Avg	Gemini Avg	Winner
1	Largest population country	4.2	3.6	GPT
2	Capital of South Korea	4.4	4.6	Gemini
3	Eiffel Tower country	4.6	3.2	GPT
4	US financial hub	3.8	3.4	GPT
5	Sahara Desert country	3.8	3.0	GPT
(Query + LLM, SimCSE)		Gemini	GPT	
Similarity (100 avg)		0.8023	0.7926	
Likert (AI)		3.36	4	
Likert (Human)		3.28	3.92	

This ablation demonstrates that the choice of reasoning-generating LLM materially affects retrieval performance, even when the downstream embedding model and corpus remain fixed. GPT’s reasoning outputs achieve greater semantic precision and relational correctness, producing higher similarity scores and superior Likert evaluations. While Gemini occasionally excels in isolated factual tasks, GPT provides more stable on-target reasoning sentences across geographically grounded queries. Accordingly, GPT serves as the default LLM for reasoning generation in our final system configuration.

7. Future Implication

While this project validated the reasoning-based search framework using a geographic corpus, the underlying methodology is domain-agnostic. The "Query + Reasoning" approach is particularly promising for specialized fields like **legal or medical retrieval**, where users often describe symptoms or legal situations without knowing the exact terminology. Future work will involve testing this pipeline on technical documentation to verify its scalability across diverse semantic landscapes.

This project demonstrated that integrating LLM-based reasoning with dense retrieval significantly outperforms traditional search methods. By transforming vague user intents into explicit declarative statements, our system bridges the gap between how humans remember information and how machines index it. We achieved a Likert accuracy score of 4.0, proving that **"Reasoning-Based Navigation"** is a viable and superior alternative to simple keyword matching for intelligent document exploration.

9. Lessons We’ve Learned

Looking back at the project as a whole, several issues repeatedly surfaced, and together they shape a clearer understanding of what can—and cannot—be expected from a reasoning-based semantic retrieval system.

First, we realized how critical evaluation really is. Early on, we assumed that high cosine similarity would naturally correlate with good answers, but throughout the experiments we saw countless counterexamples. Sentences with very high similarity sometimes had little to do with the actual query, while more relevant sentences occasionally scored lower. This gap between numerical scores and semantic correctness made it obvious that similarity alone is not a reliable indicator

of performance, and that any retrieval system grounded in embeddings must be evaluated with more than one metric.

Second, the project exposed an unexpected level of unpredictability within the dataset itself. Even within a domain as structured as Wikipedia geography articles, documents we expected to exist were missing, and concepts that felt straightforward to humans appeared in multiple forms—"ROK," "Republic of Korea," and "South Korea," for instance, all being used interchangeably. They reminded us that large, collaborative datasets are rarely as uniform or predictable as they seem.

Third, we learned that more advanced techniques do not automatically produce better results. SimCSE outperformed MPNet in many of our experiments, but not because it was universally superior. Instead, each model behaved differently depending on the type of query, the phrasing of the LLM’s reasoning step, and even the presence or absence of specific lexical cues. Likewise, GPT and Gemini produced different styles of reasoning that affected retrieval in subtle but meaningful ways. No single "best" model consistently dominated; performance depended heavily on context, and sometimes older or simpler approaches held up better than expected.

Determining the appropriate number of items to retrieve from the semantic space proved more difficult than expected. Retrieving too few candidates risks overlooking meaningful alignments, while retrieving too many introduces noise that dilutes the evaluation signal. Compounding this challenge, we observed that cosine similarity does not correlate perfectly with answer accuracy; semantically close sentences can still be factually incorrect or partially aligned. To overcome this, we had to see a wider scale distribution of the answers, but we couldn’t reach there. This revealed that retrieval depth and cosine-based scoring are both sensitive parameters rather than reliable absolutes and tuning them carefully became an essential lesson in refining our evaluation methodology.

Taken together, these observations lead to a somewhat sobering but honest conclusion. Even with careful preprocessing, multiple embedding models, LLM-generated reasoning, and a tailored evaluation framework, semantic retrieval does not behave in a fully predictable or controllable way. Each layer—dataset, model, similarity space, and evaluation—adds its own uncertainty, and their interactions only amplify that unpredictability. In the end, the system works, but never quite in the way we intend.